



Builder e Prototype

Padrões de Projeto - Aula #07

Prof. Dr. Carlos Melo

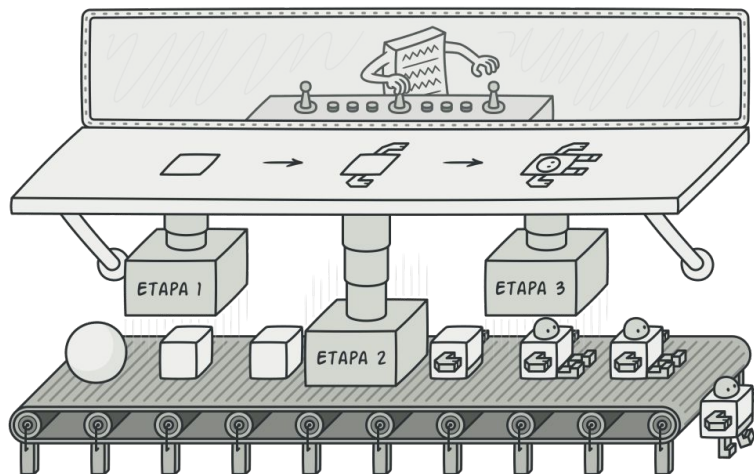
 carlos.melo@upe.br



Padrões Criacionais (parte #3)

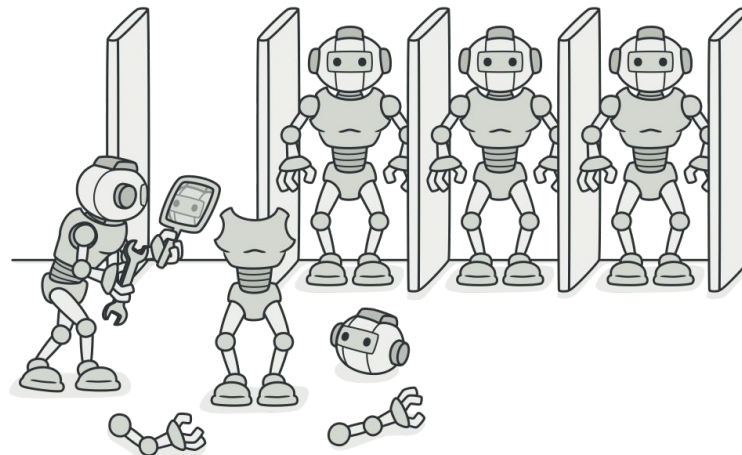
Builder

Construa um objeto complexo passo a passo, com o mesmo processo gerando representações diferentes



Prototype

Crie novos objetos copiando um existente, em vez de instanciar do zero



- As fábricas respondiam a pergunta sobre que classe instanciar
- Builder nos diz como montar um objeto, enquanto o Prototype vai dizer de onde vem



Objetivos de aprendizagem

- Reconhecer o construtor telescópico (classe com muitos parâmetros) como um *Code Smell* e saber quando ele justifica um Builder.
- Implementar o Builder e entender o papel do Director.
- As @dataclasses e argumentos nomeados em Python resolvem boa parte dos casos de Builder, vamos ver quais elas não resolvem.
- Relação entre Prototype e Flyweight (Clonando objetos que compartilha estado).



Builder - O Problema

- O acervo cresceu e agora precisamos gerar relatórios de observação.
- Um relatório pode ter:
 - Cabeçalho, logo do observatório, sumário, seção de imagens com ou sem miniaturas, tabela de metadados, gráficos de curva de luz, rodapé, marca d'água, índice,
- Nem todo relatório tem todas essas características. São treze parâmetros...

```
1 class Relatorio:
2     def __init__(self, titulo, autor, cabecalho, logo, sumario,
3                   imagens, miniaturas, metadados, graficos,
4                   rodape, assinatura, marca_dagua, indice):
5         ...
```



Builder - O Problema

- A saída tradicional consiste em criar um construtor telescópico

```
1 # Em Java/C#, seriam vários construtores diferentes
2 # Em Python atribuímos valores default
3
4 class Relatorio:
5     def __init__(self, titulo, autor, cabecalho=True, logo=None,
6                  sumario=False, imagens=None, miniaturas=False,
7                  metadados=False, graficos=None, rodape=True,
8                  assinatura=None, marca_dagua=False, indice=False):
9         ...
```



Builder - O Problema

- Chamando todos os *Thundercats*

```
1 r = Relatorio("Observação NGC 4654", "Dra. Silva", True, "upe.png",
2               True, imagens, False, True, None, True, None, True, False)
3
4
5 # -----
6
7 r = Relatorio(titulo="...", autor="...", cabecalho=True, logo="upe.png",
8               sumario=True, imagens=imagens, miniaturas=False,
9               metadados=True, rodape=True, marca_dagua=True, indice=False)
```

- O que é aquele False na sétima posição? E o True na décima segunda?
- Trocar dois argumentos de posição não vai gerar erro de implementação, mas o relatório estará errado
- Em Python dá para usar atributos nomeados, mas ainda assim fica uma *b*.

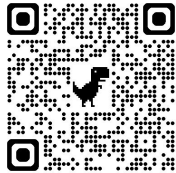


Soluções Ingênuas

- E se criássemos uma subclasse por tipo de relatório?

```
1 class RelatorioInterno(Relatorio): ...  
2 class RelatorioPublicado(Relatorio): ...  
3 class RelatorioParceiro(Relatorio): ...  
4 class RelatorioParceiroSemGraficos(Relatorio): ...  
5 class RelatorioInternoComIndice(Relatorio): ...
```

- Pode gerar explosão combinatória, com 8 opções, por exemplo, teríamos $2^8 = 256$ subclasses.
- E a diferença entre elas não é comportamento é configuração.
- Herança não deve ser usada para variar configuração.



Builder - A ideia

- Separe a construção da representação
 - Um objeto dedicado a montar e um método por etapa.

```
1 relatorio = (ConstrutorDeRelatorio("Observação NGC 4654", "Dra. Silva")
2             .com_logo("upe.png")
3             .com_sumario()
4             .com_imagens(imagens, miniaturas=True)
5             .com_metadados()
6             .com_marca_dagua("RASCUNHO")
7             .construir())
```

- Cada etapa é nomeada e etapas não usadas não aparecem.
- A ordem é livre, e o objeto só existe quando construir() é chamado.



Builder - Intenção

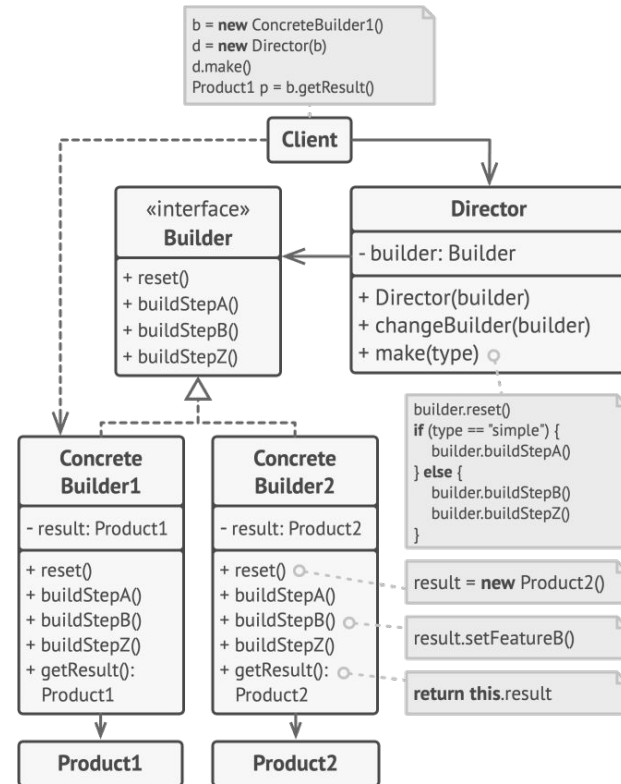
"Separar a construção de um objeto complexo da sua representação, de modo que o mesmo processo de construção possa criar diferentes representações."

- Padrão criacional
- Que separa duas responsabilidades:
 - Quais etapas e em que ordem (Director)
 - Como cada etapa é executada (Builder)
- Sem Director é o cliente que executa as etapas. É o mais comum.

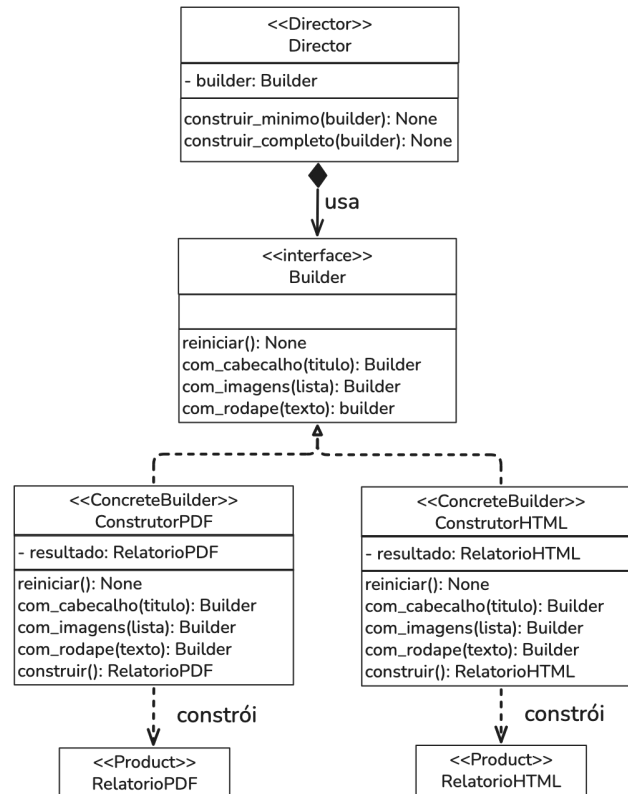
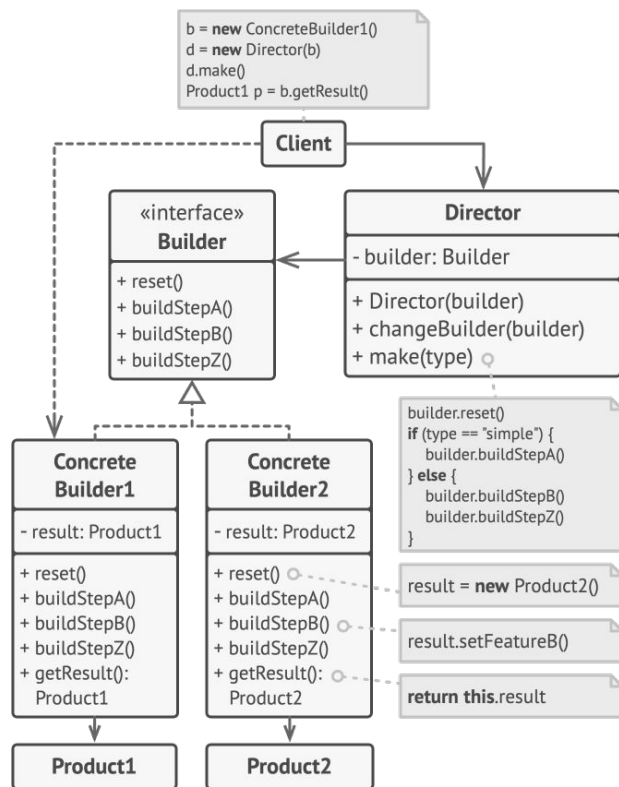
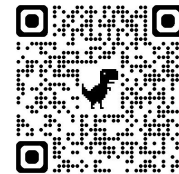


Builder - Estrutura

1. A interface **Builder** declara etapas de construção do produto que são comuns a todos os tipos de builders.
2. **ConcreteBuilders** fornecem diferentes implementações das etapas de construção e podem produzir produtos que não seguem a interface comum.
3. **Products** são os objetos resultantes.
4. **Director** define a ordem na qual as etapas de construção são chamadas.
5. **Client** associa um dos objetos builders com o diretor.



Aplicando ao nosso domínio





Aplicando ao nosso domínio

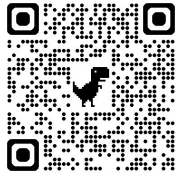
- O Director é opcional, sem o Director ficaria assim:

```
1 relatorio = (ConstrutorPDF()  
2               .com_cabecalho("NGC 4654")  
3               .com_imagens(imagens)  
4               .com_rodape("UPE Garanhuns")  
5               .construir())
```

- Com o Director podemos ter “pré-moldados”

```
1 class DiretorDeRelatorio:  
2     def relatorio_minimo(self, b: Builder, titulo: str):  
3         b.reiniciar()  
4         b.com_cabecalho(titulo).com_rodape("UPE Garanhuns")  
5  
6     def relatorio_completo(self, b: Builder, titulo: str, imagens):  
7         b.reiniciar()  
8         (b.com_cabecalho(titulo).com_sumario().com_imagens(imagens)  
9          .com_metadados().com_indice().com_rodape("UPE Garanhuns"))
```

```
1 diretor = DiretorDeRelatorio()  
2  
3 pdf = ConstrutorPDF()  
4 diretor.relatorio_completo(pdf, "NGC 4654", imagens)  
5 arquivo_pdf = pdf.construir()  
6  
7 html = ConstrutorHTML()  
8 diretor.relatorio_completo(html, "NGC 4654", imagens)  
9 arquivo_html = html.construir()  
10  
11 # Use o director quando a sequência de etapas se repete
```



Aplicando ao nosso domínio

```
1 from dataclasses import dataclass, field
2
3 @dataclass
4 class RelatorioPDF:
5     paginas: list[str] = field(default_factory=list)
6
7     def renderizar(self) -> str:
8         return "\n".join(f"--- pág {i+1} ---\n{p}"
9                             for i, p in enumerate(self.paginas))
10
11
12 @dataclass
13 class RelatorioHTML:
14     tags: list[str] = field(default_factory=list)
15
16     def renderizar(self) -> str:
17         return "<html>\n" + "\n".join(self.tags) + "\n</html>"
```

- Os dois produtos são classes independentes, já que não herdam de nada em comum, e seus métodos internos são diferentes (paginas vs tags).
- Isso é permitido e é a principal diferença para o Abstract Factory, onde os produtos precisavam compartilhar interface.

Aplicando ao nosso domínio



```
1 from abc import ABC, abstractmethod
2
3 class ConstrutorDeRelatorio(ABC):
4     @abstractmethod
5     def reiniciar(self) -> None: ...
6     @abstractmethod
7     def com_cabecalho(self, titulo: str) -> "ConstrutorDeRelatorio": ...
8     @abstractmethod
9     def com_imagens(self, imagens: list[str]) -> "ConstrutorDeRelatorio": ...
10    @abstractmethod
11    def com_rodape(self, texto: str) -> "ConstrutorDeRelatorio": ...
```

- **construir()** reinicia o builder, para que a mesma instância possa montar o próximo produto sem contaminação.
- Cada método devolve **self** é o que torna possível compor objetos.

```
1 class ConstrutorPDF(ConstrutorDeRelatorio):
2     def __init__(self):
3         self.reiniciar()
4
5     def reiniciar(self) -> None:
6         self._r = RelatorioPDF()
7
8     def com_cabecalho(self, titulo):
9         self._r.paginas.append(f"[LOGO UPE]\n{titulo.upper()}")
10        return self
11
12    def com_imagens(self, imagens):
13        for img in imagens:
14            self._r.paginas.append(f"[figura: {img}.fits]")
15        return self
16
17    def com_rodape(self, texto):
18        self._r.paginas.append(f"{'_' * 30}\n{texto}")
19        return self
20
21    def construir(self) -> RelatorioPDF:
22        resultado, self._r = self._r, RelatorioPDF()
23        return resultado
```

Aplicando ao nosso domínio

```
1 class ConstrutorHTML(ConstrutorDeRelatorio):
2     def __init__(self): self.reiniciar()
3     def reiniciar(self): self._r = RelatorioHTML()
4
5     def com_cabecalho(self, titulo):
6         self._r.tags.append(f' <h1 class="titulo">{titulo}</h1>')
7         return self
8
9     def com_imagens(self, imagens):
10        for img in imagens:
11            self._r.tags.append(f' ')
12        return self
13
14    def com_rodape(self, texto):
15        self._r.tags.append(f" <footer>{texto}</footer>")
16        return self
17
18    def construir(self) -> RelatorioHTML:
19        resultado, self._r = self._r, RelatorioHTML()
20        return resultado
```

```
1 imagens = ["hubble_001", "andromeda"]
2
3 for construtor in (ConstrutorPDF(), ConstrutorHTML()):
4     r = (construtor
5         .com_cabecalho("Observação NGC 4654")
6         .com_imagens(imagens)
7         .com_rodape("UPE Garanhuns")
8         .construir())
9     print(r.renderizar(), "\n")
```

```
1 --- pág 1 ---
2 [LOGO UPE]
3 OBSERVAÇÃO NGC 4654
4 --- pág 2 ---
5 [figura: hubble_001.fits]
6 --- pág 3 ---
7 [figura: andromeda.fits]
8 --- pág 4 ---
9 -----
10 UPE Garanhuns
11
12 <html>
13 <h1 class="titulo">Observação NGC 4654</h1>
14 
15 
16 <footer>UPE Garanhuns</footer>
17 </html>
```



Aplicando ao nosso domínio

- Builder com validação no método de **construir()**
- Somente objetos válidos poderão ser construídos

```
1 class ConstrutorPDF(ConstrutorDeRelatorio):
2     def __init__(self):
3         self.reiniciar()
4
5     def reiniciar(self):
6         self._r = RelatorioPDF()
7         self._tem_cabecalho = False
8         self._tem_imagens = False
9         self._miniaturas = False
10
11     def com_miniaturas(self):
12         self._miniaturas = True
13         return self
14
15     def construir(self) -> RelatorioPDF:
16         if not self._tem_cabecalho:
17             raise ValueError("relatório exige cabeçalho")
18         if self._miniaturas and not self._tem_imagens:
19             raise ValueError("miniaturas exigem imagens")
20         resultado, _ = self._r, self.reiniciar()
21         return resultado
```

```
1 ConstrutorPDF().com_miniaturas().construir()
2 # ValueError: miniaturas exigem imagens
```

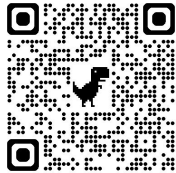



Alternativa Pythônica

- Boa parte do problema desaparece com recursos da linguagem:

```
1 from dataclasses import dataclass, field
2
3 @dataclass(kw_only=True) # proíbe argumentos posicionais
4 class Relatorio:
5     titulo: str
6     autor: str
7     logo: str | None = None
8     sumario: bool = False
9     imagens: list[str] = field(default_factory=list)
10    miniaturas: bool = False
11    marca_dagua: str | None = None
12
13    def __post_init__(self):
14        if self.miniaturas and not self.imagens:
15            raise ValueError("miniaturas exigem imagens") # validação
```

```
1 r = Relatorio(titulo="NGC 4654", autor="Dra. Silva",
2               sumario=True, imagens=["hubble_001"], miniaturas=True)
3
4 Relatorio("NGC 4654", "Dra. Silva") # TypeError
```



Alternativa Pythônica

- Mas não resolve tudo:
 - Não é possível construir o objeto em etapas por meio de dataclasses
 - Todos objetos vão ter a mesma cara-crachá
 - Se o objeto pode ser construído de uma vez, use `@dataclass(kw_only=True)`.



Prós e Contras do Builder

Prós

- Constrói objetos passo a passo, útil quando os dados chegam aos poucos.
- O mesmo processo gera representações diferentes.
- Respeita o **S** do **SOLID**, a lógica de montagem sai da classe do produto.
- O produto nunca existe em estado inválido ao se adicionar validações no construir()

Contras

- Aumenta a quantidade de classes: interface + builders + director + produtos.
- Só se paga quando o objeto é realmente complexo.
- Em Python, `@dataclass(kw_only=True)` cobre a maioria dos casos com zero classes extras.

Não use:

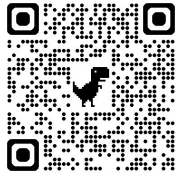
- Se o sistema é pequeno.



Builder x Abstract Factory

Os dois criam objetos compostos, porém...

	Builder	Abstract Factory
Foco	Como montar, em etapas	Qual família instancia
Produtos	Podem ser sem relação entre si	Precisam de uma interface comum
Devolve	Ao final, via construir()	Imediatamente, a cada chamada
Ênfase	Processo de construção	Consistência entre os produtos



Prototype - O Problema

- Você tem um objeto e quer uma cópia exata dele. Como que você faz?

```
1 copia = Forma()  
2 copia.x = original.x  
3 copia.y = original.y  
4 copia.cor = original.cor
```

- Parece que foi... porém não.
- Nem todo campo é visível de fora e só é possível copiar o que se enxerga

```
1 class Circulo(Forma):  
2     def __init__(self, raio):  
3         self.raio = raio  
4         self.__cache_area = 3.14159 * raio ** 2  
5         self.__id_render = _proximo_id()
```



Prototype - O Problema

- Além disso, copiar geralmente exige conhecer a classe concreta

```
1 def duplicar(forma: Forma) -> Forma:  
2     nova = Circulo(...)  
3     ...
```

- Mas e se vier um retângulo?
 - if, elif, else...



Prototype - A ideia

- Se o problema é que de fora não dá para copiar, então delegue a cópia para dentro.

```
1 copia = original.clonar()
```

- O objeto se copia a si mesmo
 - Ele enxerga os próprios campos privados.
- O cliente chama clonar() pela interface e não precisa saber se é Circulo ou Retangulo.
- O objeto que suporta clonagem é o protótipo.

```
1 def duplicar_todas(formas: list[Forma]) -> list[Forma]:  
2     return [f.clonar() for f in formas]
```



Prototype - Intenção

"Especificar os tipos de objetos a serem criados usando uma instância protótipo, e criar novos objetos pela cópia desse protótipo."

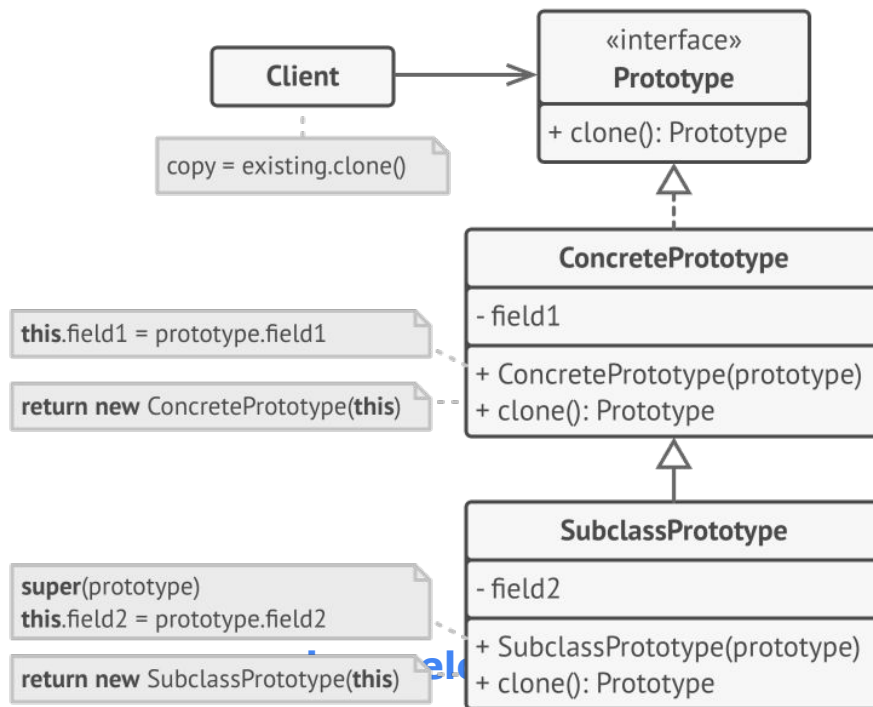
- As vezes é mais barato clonar que construir (objetos que são frutos do resulta de cálculo entre matrizes, por exemplo, a cópia já traz o resultado calculado anteriormente)

	Cria objeto a partir de
Factory Method / Abstract Factory	Uma classe
Builder	Etapas
Prototype	Outro objeto



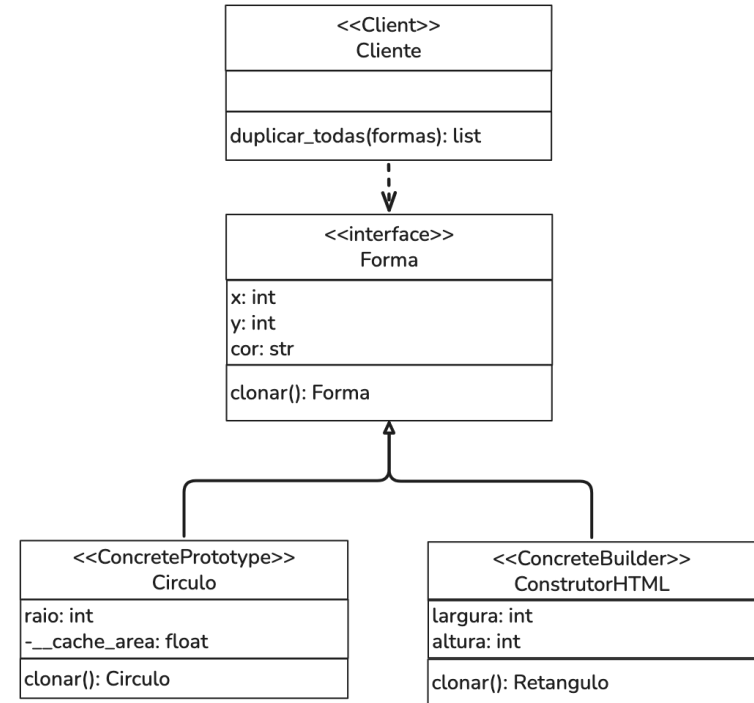
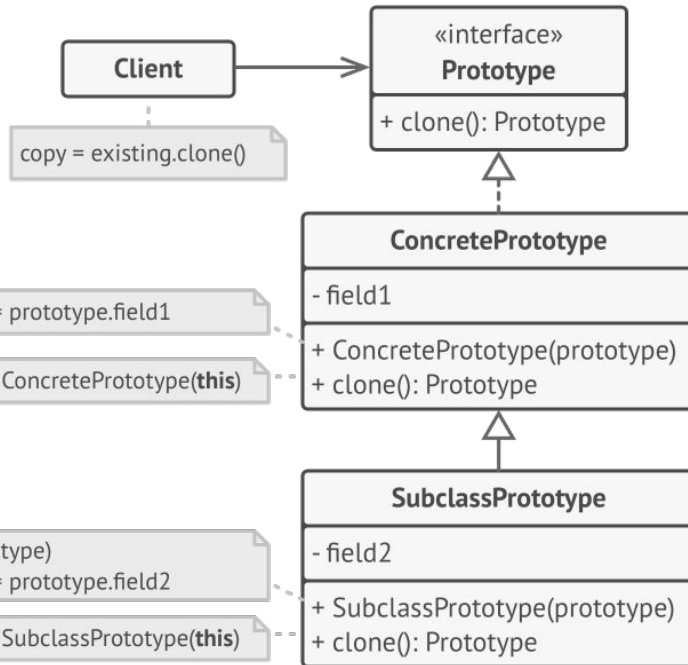
Prototype - Estrutura

1. **Prototype** declara os métodos de clonagem. Geralmente o **clone()**.
2. **ConcretePrototype** implementa o método de clonagem.
3. O **Client** pode produzir uma cópia de qualquer objeto que segue a interface do protótipo





Aplicando ao nosso domínio



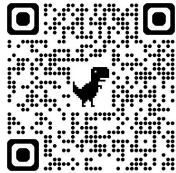


Aplicando ao nosso domínio

```
1 import copy
2
3 class Forma:
4     def __init__(self, x: int = 0, y: int = 0, cor: str = "preto"):
5         self.x, self.y, self.cor = x, y, cor
6
7     def clonar(self) -> "Forma":
8         return copy.copy(self)
9
10    def __str__(self):
11        return f"type(self).__name__({self.x},{self.y},{self.cor})"
12
13
14    class Circulo(Forma):
15        def __init__(self, raio: int, **kw):
16            super().__init__(**kw)
17            self.raio = raio
18            self.__cache_area = 3.14159 * raio ** 2
19
20        def area(self) -> float:
21            return self.__cache_area
```

```
1 original = Circulo(raio=10, x=5, y=3, cor="azul")
2 copia = original.clonar()
3
4 print(copia, copia.area())
5 print(type(copia).__name__)
6 print(copia is original) # False
```

- O círculo sequer é mencionado no cliente. Atributos privados foram incluídos.



Mas cuidado com a cópia rasa

```
1 class Poligono(Forma):
2     def __init__(self, vertices: list[tuple], **kw):
3         super().__init__(**kw)
4         self.vertices = vertices
5
6 p1 = Poligono(vertices=[(0,0), (1,0), (0,1)], cor="verde")
7 p2 = p1.clonar()
8
9 p2.vertices.append((1,1))
10
11 print(p1.vertices)
12 print(p2.vertices)
13 print(p1.vertices is p2.vertices)  # True
```

- Cópia rasa duplica referências, mas não o conteúdo. O que não é um problema para tipos imutáveis. Porém para os mutáveis é o caos instaurado.



E a cópia profunda? (Deepcopy)



```
1 def clonar(self) -> "Poligono":  
2     return copy.deepcopy(self)
```



```
1 p2 = p1.clonar()  
2 p2.vertices.append((1,1))  
3 print(p1.vertices)  
4 # [(0,0), (1,0), (0,1)]
```

- É mais cara, mas copia tudo. Não é ideal para o Flyweight, por exemplo, quando a cópia rasa é uma melhor opção (já que queremos reaproveitar as referências).
- Além disso, o deepcopy deve ser usado com cuidado em alguns objetos, sobretudo recursos compartilhados, como uma conexão ao banco, por exemplo (veremos o singleton na próxima aula)



E a cópia profunda? (Deepcopy)

```
1 class Documento:
2     def __init__(self, conexao_bd, texto):
3         self.conexao = conexao_bd
4         self.texto = texto
5
6     def __deepcopy__(self, memo):
7         novo = Documento.__new__(Documento)
8         novo.conexao = self.conexao
9         novo.texto = copy.deepcopy(self.texto, memo)
10        return novo
```

- Todavia ainda é possível personalizar o que estamos *deepcopyando* ao passar por parâmetro o que queremos copiar na íntegra e aquilo que queremos referenciar, como a conexão, fica de fora.



Prós e Contras do Abstract Factory

Prós

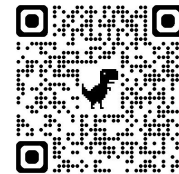
- Clona objetos sem acoplar o cliente às classes concretas.
- Copia estado privado, que o cliente não conseguiria alcançar.
- Elimina código de inicialização repetitivo partindo de um *preset* pronto.
- Alternativa a hierarquias de fábricas quando as variações são de configuração, não de comportamento.
- Pode ser mais rápido que construir, quando há cálculo caro envolvido.

Contras

- Clonar objetos com referências circulares exige cuidado
- A escolha rasa/profunda é sutil e o erro é silencioso
- Em Python, `copy.copy` e `copy.deepcopy` já existem, logo a interface **clonar()** às vezes é cerimônia desnecessária.

Não use:

- Objetos baratos de construir, sem estado privado relevante, e cujo tipo o cliente já conhece. Nesse caso, `Circulo(raio=10)` é mais claro que `proto.clonar()`.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br